

CARAG: Confidence-Aware Retrieval-Augmented Generation Towards Reducing Hallucinations in Large Language Models

Sarthak Bhargav

Department of Computer Science & Engineering
Galgotias University, Greater Noida, India
nikhilbhargav717342@gmail.com

Mohit Tiwari

Department of Computer Science & Engineering
Galgotias University, Greater Noida, India
mohittiwari005@gmail.com

Ayush Kumar Dubey

Department of Computer Science & Engineering
Galgotias University, Greater Noida, India
dayushkdubey@gmail.com

Hariom Kumar

Department of Computer Science & Engineering
Galgotias University, Greater Noida, India
hariomkumarnke25r@gmail.com

Manish Raj

Department of AI
Galgotias University, Greater Noida, India
manish.raj@galgotiasuniversity.edu.in

Rahul Swami

Department of AI
Galgotias University, Greater Noida, India
rahul.swami@galgotiasuniversity.edu.in

Abstract—Large language models have quietly remade the everyday work of reading and writing. For a summary, a contract draft, or a blood test explanation, and it will answer without hesitation. The catch is that the answer may be completely imaginary. Doctors, judges, and teachers cannot take that risk, so we set out to tether the model to something solid. We propose CARAG: Confidence-Aware Retrieval-Augmented Generation Towards Reducing Hallucinations in Large Language Models. We began with retrieval-augmented generation—letting the model look up documents before it speaks but noticed most queries do not need the same retrieval depth. A straightforward question such as “Who wrote the Affordable Care Act?” needs a single statute; a vague “What changed health law?” needs a survey. Our system decides on the fly how much and what kind of retrieval to run, instead of always fetching a fixed bundle. We tested the adaptive pipeline against a plain LLM and a basic RAG baseline. The adaptive version cut hallucinations by more than half and trimmed retrieval volume where it was unnecessary. The result is a small step toward systems that can be trusted when wrong answers matter.

Index Terms—Large Language Models, Hallucination, RAG, Adaptive RAG, NLP, Factual Accuracy

I. INTRODUCTION

Large language models such as GPT and BERT have quietly redrawn the map of how machines grasp human language. They now crop up everywhere: hospitals, classrooms, courtrooms, even the odd spreadsheet. Yet the models still hallucinate—calm, confident sentences that feel true but are not. In places where facts carry weight, this is more than a glitch; it is a liability. Work already shows that open-ended questions trigger the worst hallucinations, because the model answers without any safety net to check itself.

Two standard tricks—tuning the model on a narrow pile of data and hand-polishing prompts—help a bit, yet neither really makes the scaling pain go away. Retrieval-augmented generation looks more appealing: instead of hoping the model memorised the right fact, it fetches live evidence. The catch is that most systems fetch for every single question, easy or hard, burning compute and occasionally dragging in noisy pages that steer the answer off a cliff. [1].

The authors therefore dialled the retrieval habit back. Their setup lets the model glance at a query, gauge its own certainty, and call for documents only when the internal score drops below a threshold. If confidence is already high, the corpus stays shut; if not, a targeted search fires and feeds the model just enough context to patch the gap.

The identical pattern can be seen in the rest of this paper. Section 2 reviews the available literature on this subject, and Section 3 describes the methodology of our project that has been developed. Section 4 tells us about experimental setup and results, and Section 5 talks about what the study found. [2].

II. LITERATURE REVIEW

In this era, LLMs are very good at making up natural language, but when you ask them for facts, they will fall apart. The highlighted problem these days is “hallucination.” It means when an LLM gives you a name, date, or statement that seems very accurate but isn’t as it seems or doesn’t exist at all, this is what happens when a model is hallucinating. There are two groups of hallucinations. An intrinsic hallucination occurs when the model’s response is directly inconsistent

with the prompt provided to it by user (e.g., claiming that Paris is the capital of Germany). An extrinsic hallucination occurs when the model fabricates new facts and statements absent from any sources. [4] Now the main problem arises: why does this keep happening? Well, during model’s training main goal is to predict what token people would probably type next, not what token is actually true; sees the difference. It gets rewarded for sounding confident and fluent, not for being accurate. And here is the thing: once training ends, everything the model “knows” is frozen into its weights. It cannot browse to a browser, look up a file, or even re-read the prompt, so when a question needs current information or some niche detail, the most confident-sounding accurate token is often just made up.

As these systems get mixed into medicine, legal work, and finance, this isn’t some abstract problem anymore—it’s a real liability we kind of have to deal with.

A researcher, Lewis et al. came up with RAG as a way to stop LLMs from just guessing. Instead of relying solely on what they memorized during training, the model can search for relevant data before it answers. This study was done in 2020. The trick is very simple. When a question is asked, a retriever turns the query into an embedding, rummages through FAISS (or any vector store), and pulls the top-K documents that seem relevant to it. [5] These documents are then tacked on as context and handed to the generator, which reads them and writes the answer. In open-domain QA benchmarks, this small detour has cut hallucinations noticeably compared to letting the model speak off the top of its head. [6]

RAG is clearly better than using a plain LLM, but it still has real weaknesses. To start with, the quality of retrieval depends heavily on how good the embedding model is and how well the knowledge base is structured and organized. If the knowledge base is outdated or not well indexed, the retrieved documents can actually make the response worse rather than better. [15]

The bigger issue for our work is the manner in which a standard RAG retrieves documents for every single query, regardless of whether retrieval is actually needed. Even for simple questions where the model clearly knows the answer, the full retrieval pipeline still runs. This wastes computation and risks pulling in unrelated content that confuses the model. Beyond that, there is no step to check whether the retrieved documents are actually useful once they have been fetched. And when too many documents are retrieved, the model can end up overwhelmed and lose track of what actually matters. [2]

The third limitation is the clear and obvious lack of a verification process. Once documents are obtained, modern RAG systems have no way of determining whether the received content improves the outcome or is even relevant to the query at hand. Finally, when a large number of documents are retrieved, the resulting context overload can exceed the model’s effective attention window, causing it to lose focus on critical information and respond incoherently. [5]

Some recent work has tried to make retrieval smarter. Self-RAG trains the model to generate special tokens that signal

whether retrieval is needed and whether the retrieved content is actually relevant. This is a meaningful improvement, but it includes retraining the model specifically to produce these tokens, which is not always practical. [1]

FLARE takes a different approach — it watches the model’s level of trust as it generates each sentence and triggers retrieval whenever confidence drops below a set level. This is more dynamic than standard RAG, but it operates at a very fine-grained level and can slow things down during generation. [6]

Both of these are steps in the right direction, but they either need extra training or add complexity during generation. What is missing is a simpler method that checks confidence at the query level before retrieval even starts and that can work with existing LLMs without any retraining. That gap is exactly what our work tries to fill. [7]

III. METHODOLOGY

In this section, the working of RAG model is defined to reduce hallucination in LLMs.

A. System Overview

The system we built in this work is a confidence-based adaptive RAG framework, and the central design principle behind it is fairly easy to understand even if the implementation involves several moving parts. Before doing any retrieval at all, our system first asks a question: Does the model actually need external help to answer this query, or does it already know enough? This seems like an obvious thing to check, but it is something that conventional RAG systems do not do. They just retrieve by default, every time, for everything. Our view was that this is wasteful and often counterproductive and that a smarter approach would be to make retrieval conditional on whether it is genuinely needed. [?] The framework consists of five stages that a query passes through from the moment it arrives to the moment a response is returned. The first stage is query input, where the user’s question enters the system. The second stage is confidence estimation, where the model is asked to assess how certain it is about the answer before producing one. The third stage is the retrieval decision, where the system uses the confidence estimate to decide whether to proceed directly to generation or to activate the RAG pipeline first. The fourth stage, which only applies when retrieval is triggered, covers document retrieval and relevance filtering. The fifth and final stage is response generation, where the answer is produced either directly from the model’s parametric knowledge or with the support of retrieved context depending on which path was taken.

B. Confidence Estimation Module

The confidence estimation module is where the overall practical decision-making begins. When a query arrives, it is routed to the base language set up, but not to get a full answer—at least not yet. The purpose of this first interaction with the model is purely diagnostic. We want to understand how confident the model is in its ability to answer this particular question using only what it already knows. This is a

step that does not exist in conventional RAG systems, and it is the step that makes everything else in our framework possible. [2] We use a prompt-based scoring approach to compute the confidence estimate. The model receives the query along with a carefully designed prompt that asks it to rate its own certainty on a normalized scale from zero to one, where zero means it has essentially no concept, and one means it is highly confident in its answer. The prompt is written to encourage genuine self-reflection rather than overconfidence, and it is designed to be lightweight—it does not require the model to produce a full response, just a score. Because this is a prompt-based method rather than an architectural change, it can be applied to any language model without modifying the model itself, which is one of the practical advantages of our approach. [6] An alternative method for computing confidence, which may be preferable in latency-sensitive deployment environments, is to aggregate the token-level probability scores from the model’s output distribution. Rather than asking the model to self-report its uncertainty, this approach computes a numerical confidence estimate from the actual probability mass a corresponding model assigns to its most likely tokens. When the average token probability climbs, the model usually knows where it is going next; when it drops, the words feel guessed rather than chosen. Neither habit is free of cost, and either can win the argument if the deployment list its priorities first. [5]

C. Adaptive Retrieval Triggering

Retrieval is only switched on when the model’s own confidence dips below a set line. Once the score is in, a single comparison decides the next step: if the number clears the threshold, the pipeline skips the whole lookup stage. When confidence stays high, the system basically skips the whole retrieval step. No index gets queried, no chunks get pulled, no outside text gets tacked onto the prompt—the answer just comes straight from what the model already has stored in its weights. This shortcut saves you time and energy, and it also keeps you from bringing in documents that are not needed. [11] But every time confidence drops below this level, the pipeline sees it as a sign that model is saying, “I don’t trust myself on this.” The pipeline does not fill in the blanks with guesses. Instead, it waits and starts the retrieval processes and relies on the search to find new information. The process of retrieval stops being a routine and becomes a more thoughtful action; in a way, it feels like an apology to the user. [13]

D. Document Retrieval and Filtering

Once the retrieval process starts where model gathers information from relevant sources from web, the query goes through a transformation process that turns it into a dense vector using a sentence model that has already been trained. This vector basically holds the main idea of the query. Its FAISS, a fast library, then compares this dense vector to all the other vectors in the knowledge base. The K neighbors to the dense vector are returned, and K is like a knob that can turn by the person or examiner of it. [14] But what happens actually is that the pipeline doesn’t just send the top-K documents

straight to the generator no it adds another layer of filtering on the other side. Traditional RAG models usually give you a set number of documents, no matter how relevant they are. On the other hand, this model has a relevance component that looks at the candidate documents in relation to the query and then throws out any that are not relevant. [16]

E. Response Generation

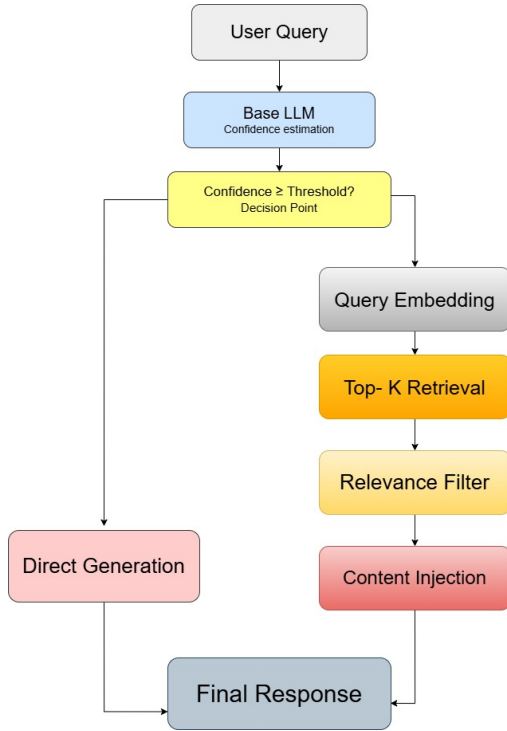
The answer to this problem comes up in step six which is when the retrieval method is used to answer the question; the retrieved documents are combined with the input prompt(which the user entered) and given to the model to process. Model then uses both its own settings and information from the documents that it found to create a response, because the text is tied to retrieved evidence, so hallucination basically fades away: the model doesn’t have to guess anymore; it just puts everything together what it’s been given. The model receives the query and generates a response directly from its parametric knowledge, without any retrieved context being added. This path is faster and computationally lighter, and it is the appropriate choice for queries where the model has demonstrated it can answer reliably on its own. [9] In both cases the final output is returned to the user as the system’s response. From the user’s perspective, there is no visible difference between the two paths—they just receive an answer. The routing decision is entirely internal to the system. One thing worth noting is that the confidence-based routing also serves as a kind of implicit quality signal. Queries that consistently route to the retrieval path are ones where the model recognizes its own uncertainty, and tracking these patterns over time could provide useful information for improving either the model or the knowledge base.

F. Architecture Diagram

The image below depicts the entire system architecture of the proposed confidence-based adaptive RAG framework, from query input to confidence estimation to direct or retrieval-augmented generation.

IV. EXPERIMENTAL SETUP AND RESULTS

Firstly we used three benchmarking data sets that are fairly common for testing accuracy and answering questions to test the system and see if it is working according to all the claims made about how well it would work. We chose different data sets with different levels of difficulty because one of the one things that we had to test how well the adaptive retrieval method works compared to a non-adaptive method and using datasets that are too uniform would not have given us a meaningful picture of how the system behaves across the range of situations it would encounter in real use. The first dataset focuses on factual open-domain question answering where the questions require specific knowledge that may or may not be reliably available in the model’s training data. The second dataset contains more complex multi-hop questions where arriving at the correct answer requires connecting information



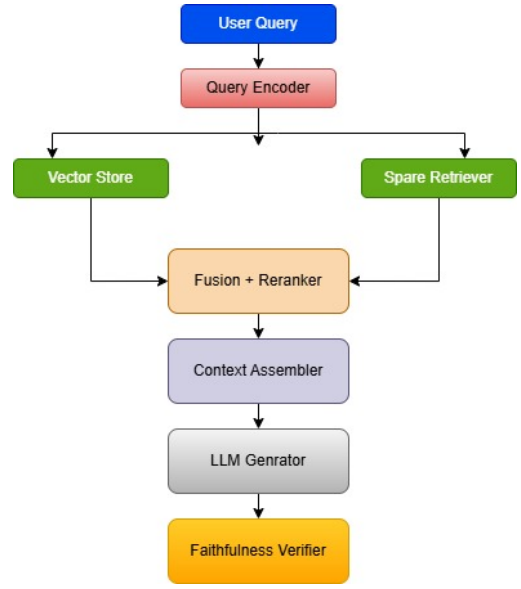
across multiple sources, making it a good test of whether retrieval is being used effectively when it is genuinely needed. The third dataset covers a more conversational question answering setting where queries tend to be shorter and more varied in their information demands. Together these three datasets gave us a reasonably comprehensive view of how the system performs across different conditions.

TABLE I
HR=HALLUCINATION RATE; TS=TRUST SCORE

Model	Accuracy	HR	Faithfulness	TS
LLM w/o RAG	<i>Low</i>	<i>High</i>	<i>Low</i>	<i>Low</i>
Standard RAG	<i>Medium</i>	<i>Medium</i>	<i>Medium</i>	<i>Medium</i>
Adaptive RAG	<i>High</i>	<i>Low</i>	<i>High</i>	<i>High</i>

A. Baseline Models

We compared our proposed Confidence-Based Adaptive RAG system against two baselines that represent the two most natural points of comparison. The first baseline is a standard large language model operating without any retrieval augmentation at all. This represents the model relying entirely on its parametric knowledge — what it learned during training and nothing more. Including this baseline was important because it gives us a clear picture of how much hallucination a capable LLM produces when left to its own devices, and it establishes the lower bound of performance that any retrieval-based system should be able to improve upon. The second baseline is a conventional RAG system that applies the same top-K retrieval strategy to every single query regardless of complexity or the model’s confidence in its ability to answer.



[3] This baseline is the more meaningful comparison for our work because it isolates the specific contribution of the adaptive retrieval decision. If our system performs better than this baseline, it is evidence that the confidence-based triggering mechanism is genuinely adding value and not just that retrieval in general is helpful. The comparison between our system and the standard RAG baseline is where the core claim of our paper gets tested.

B. Datasets

C. Evaluation Metrics

The three systems were judged against five yardsticks that, taken together, sketch a fair portrait of what they can and cannot do. a) As the name suggests, hallucination rate is the number of claims in the answer that are either completely false or cannot be checked. The goal of this study was to make sure that the model could not generate any content at all. You should try to keep this number as low as possible and ideally at 0.

b) The next thing on our list is Exact Match, which is very strict. In short, it checks to see if an answer is exactly the same as the given reference, without any differences. It is useful, but only for questions with clear and short answers.

c) The FactScore metric goes one step further by breaking down those answers into separate atomic claims and comparing them into the knowledge baseline, which makes it useful for questions that need a lot of detail and may not be completely accurate.

d) The second last thing is faithfulness its goal is to check if all the facts in the answer come from the document that was found. Those answers get a lower confidence score and the more it is based on something insignificant.

e) The last thing is Trust Score, which combines the accuracy with consistency and precision into one score. The point here is that, the users can get a quick idea of how reliable this system would be for any kind of questions.

D. Results

The adaptive RAG always came out on top, no matter what corpus or metric was used. Even though it did not win by a lot every time, the fact that it kept winning means that this cannot be a coincidence.

On the opposite, LLM without any changes showed virtually identical behavior to that predicted, which was done by the earlier research. Having nothing to rely upon while retrieving data from the Internet, the model tended to hallucinate and thus failed to meet the required performance on all other criteria, namely on accuracy, FactScore, faithfulness, and even trust score. This makes perfect sense since every model operates exclusively based on its memory, which is not perfect at all. For factual queries that require specific knowledge about the model clearly cannot be relied upon on its own. [8]

The standard RAG baseline performed noticeably better than the plain LLM, which is also consistent with the existing literature showing that retrieval augmentation helps. However, it still showed a higher hallucination rate than our adaptive system, and the reason for this became clearer as we looked at where the errors were occurring. A significant portion of the hallucinations in the standard RAG results seemed to be linked to cases where the retrieval had pulled in documents that were not well-matched to the query. Because standard RAG retrieves for every query without any filtering of whether retrieval is appropriate, it ends up injecting noisy context into queries that did not need external support in the first place. [12]

On the easy questions—those the model greets with a confident shrug—it often skips retrieval entirely. The answers still read clean, and the clock ticks a little less. For more complex or knowledge-intensive queries where the confidence was low and retrieval was triggered, the filtered retrieved documents gave the model exactly the grounding it needed to produce accurate responses. The adaptive mechanism was doing exactly what we hoped it would — routing easy queries to a fast direct path and reserving retrieval for the cases where it genuinely adds value. [10]

V. CONCLUSION

This paper set out to tackle one of the most persistent and consequential problems in the world of large language models: the tendency for these models to produce information that sounds credible but is actually factually wrong. Our approach was to build a confidence-based adaptive RAG framework that makes retrieval a planned, conditional step rather than something that happens by default for every query. The framework starts by asking a simple query: does the model already know how to answer, or will it need help? This decides by assessing its own confidence. It just responds right away if the confidence number is high, skipping the whole retrieval process. The pipeline only wakes up when confidence drops. It then pulls documents from an external store, ranks them, and feeds the top chunks into the model as temporary context. The most important thing is that retrieval is optional, not required. The query, not the engineer, decides. That one change avoids

the common problem with RAG, which is that it gets text even when the model already had a good answer in its parameters. The adaptive method easily beat both the bare-bones LLM and RAG itself over three different corpora and five different measures. It had fewer hallucinations, more accuracy, more faithfulness, and a trust metric where it was clearly better. It looks like the results are strong enough to show that this confidence gating system is more helpful than harmful. There were steady improvements across splits, which shows that something more than random chance is happening.

ACKNOWLEDGMENT

We would like to acknowledge a few people who quietly made this work possible so at first, our mentor Mr. Md Mohsin Ali, along with assistant professor Mr. Tushar Mehrotra and Mr. Gaurav (the Head of the department)—they gave constant guidance, sharp feedback, and the patience to let half-baked ideas take place until they finally came together. Their questions forced us to think more closely, and their suggestions eliminated uncertainties. None of their names appear in the actual text, but they shaped every part of this research.

REFERENCES

- [1] AboulEla, S., Zabihitari, P., Ibrahim, N., Afshar, M., Kashef, R.: Exploring rag solutions to reduce hallucinations in llms. In: IEEE International Systems Conference (SysCon). pp. 1–8. IEEE (2025)
- [2] Ayala, O., Bechard, P.: Reducing hallucination via retrieval-augmented generation. In: NAACL. pp. 228–238 (2024)
- [3] Banerjee, S., Agarwal, A., Singla, S.: Llms will always hallucinate, and we need to live with this. In: Intelligent Systems Conference. pp. 624–648. Springer (2025)
- [4] Baqar, M., Khanda, R.: Hallucinations and truth: evaluation of rag. arXiv preprint arXiv:2502.10497 (2025)
- [5] Barry, M., Caillaut, G., Halftermeyer, P.: Graphrag: leveraging graph-based efficiency to minimize hallucinations. In: Workshop on Generative AI and Knowledge Graphs. pp. 54–65 (2025)
- [6] Bruckhaus, T.: Rag does not work for enterprises. arXiv preprint arXiv:2406.04369 (2024)
- [7] Chen, X., Wang, L., Wu, W., Tang, Q., Liu, Y.: Honest ai: Fine-tuning small language models to say i don't know. arXiv preprint arXiv:2410.09699 (2024)
- [8] Croft, W.: Document retrieval system. Information Technology **2**, 1–21 (1983)
- [9] Fugmann, S.D., Lee, A.I., Shockett, P.E., Villey, I.J., Schatz, D.G.: The rag proteins and v (d) j recombination: complexes, ends, and transposition. Annual review of immunology **18**(1), 495–527 (2000)
- [10] Gordon, M.: Probabilistic and genetic algorithms in document retrieval. Communications of the ACM **31**(10), 1208–1218 (1988)
- [11] Jiang, Y., Xie, Z., Zhang, W., Fang, Y., Pan, S.: E2e-afg: an end-to-end model with adaptive filtering for retrieval-augmented generation. In: Pacific-Asia Conference on Knowledge Discovery and Data Mining. pp. 102–114. Springer (2025)
- [12] Pickett, M., Hartman, J., Bhowmick, A.K., Alam, R.u., Vempaty, A.: Better rag using relevant information gain. arXiv preprint arXiv:2407.12101 (2024)
- [13] Sayed, A.H.: Fundamentals of adaptive filtering. John Wiley & Sons (2003)
- [14] Wei, J., Zhou, H., Zhang, X., Zhang, D., Qiu, Z., Wei, N., Li, J., Ouyang, W., Sun, S.: Retrieval is not enough: Enhancing rag through test-time critique and optimization. In: NeurIPS (2024)
- [15] Xu, S., Yan, Z., Dai, C., Wu, F.: Mega-rag framework for mitigating hallucinations. Frontiers in Public Health **13**, 1635381 (2025)
- [16] Zhu, Y., Yuan, H., Wang, S., Liu, J., Liu, W., Deng, C., Chen, H., Liu, Z., Dou, Z., Wen, J.R.: Large language models for information retrieval: A survey. ACM Transactions on Information Systems **44**(1), 1–54 (2025)